My teaching philosophy is closely tied to one powerful method for design: the iterative design process. This practice engages in repeated cycles of development, where identifying and then correcting for flaws or errors is essential. Similarly, the core philosophy of my teaching and mentoring is that errors are an essential part of learning. Some effective practices in iterative design include *engaging in discussion*, *providing and receiving feedback*, and *staying connected to context*; I incorporate all of these processes in my teaching and mentoring. Using these strategies helps support both students who struggle but also, counter-intuitively, the students who are strongest. Helping students develop resilience and motivation in the face of very natural failures is part of my effort to make computer science welcoming to all students. At the same time, I always try to provide sufficient challenges for outstanding students to really test their mettle – and fail too!

Teaching

Encouraging Discussion: My teaching uses engagement in the classroom to help students fully understand the material – whether that understanding is learning terms and theories at the introductory level or analyzing and critiquing approaches at the graduate level. Questions and discussion in class can also help students understand that errors are a natural part of learning. When students answer incorrectly, I first explain why the incorrect answer makes some sense, and then reinforce how that differs from the correct answer. For example, I tell students that based on their past courses, it would be natural to expect Javascript closures to pass the value of a variable or a reference to the variable, so the fact that it does not is important to pay attention to. Questioning can also also help even the strongest students stretch themselves. I follow the philosophy that *learning does not end with the right answer*: when a student answers a question correctly, I follow up with additional questions that build additional knowledge.

Students are often nervous to answer questions, particularly in large classroom settings. Knowing that incorrect answers are normalized, not stigmatized, and that I will help build students up to the correct answer helps make them more confident. However, there are many reasons students may not want to speak in class: lacking confidence in their ideas, lacking confidence in their speaking or being a non-native speaker, or being uncomfortable with conflict. In order to respect these students needs while still engaging their ideas in the classroom, I have developed a number of strategies. For example, in graduate level classes, I have students write critiques of papers or answer open-ended questions before class. Then, discussion leaders (other students and/or myself) can highlight some of those ideas as launching points for discussion. These students' voices can be heard, even if they are not comfortable introducing the ideas during class themselves.

Providing Feedback: Another way I try to ensure that students know (and deeply appreciate) that errors are an essential part of the learning process: I try to make exams very challenging. This normalizes that all students will make errors, and also allows the strongest students to really challenge themselves. In my experience, students do not have problems with these exams, as long as they know in advance how difficult they will be. To do so, I use another approach drawn from iterative design: providing feedback early and often! I like to provide practice exams, usually based on prior years' exams. I also use difficult in-class quizzes from the first or second week of the semester. In a large class, this will typically be done by showing a multiple-choice question and having students vote using their phones, laptops or other tools. This serves a number of purposes in addition to establishing expectations. It helps all students get a sense of how well they know the material, since students can feel overconfident about their understanding, until forced to make a real choice. It can prompt discussion in even a large class, around why students chose popular

incorrect choices and how that reveals misunderstandings. Finally, it is important feedback for me to evaluate my own teaching and to understand which material needs to be clarified further.

Connecting to the Broader World: Finally, I try to ensure that students understand that learning does not end when they leave the classroom; in a fast-moving field like computer science, they will need to learn from errors throughout their careers. For example, in my mentoring, office hours and lab sections, I put as much emphasis on learning *how to learn* from errors as on learning correct methods initially. This can take many forms. This can be simple, like learning how to read a "bug" or error message, and how to learn what that error message means. There are also innovative ways to connect with the field; in one case, we connected to industry practice, while also showcasing the importance of iteration in entrepreneurial settings, like identifying whether an idea for an app or startup is good. In the web development course, we did so using the students' final project. In addition to grades on the standard rubric, the projects were also evaluated by a panel of web developers working in the local research park. We hosted an after-hours event at the research park conference center, where students gave brief presentations on their projects. At the end of the night, the external panel gave awards for the best projects on several dimensions (best idea, front end design, back end, etc.) as well as feedback on concept and implementation. Students found this external validation by real people working in their field very motivating and it also recognized the work of the outstanding groups and students who went well beyond the project expectations.

Mentoring

Research: I have been fortunate to mentor over 15 undergraduate and graduate students on research that has resulted in a number of coauthored publications. I've mentored students on my projects, but also their own. For example, one semester students came back from winter break excited about a newly popular game: HQTrivia. We built and tested competing real time systems to predict answers to HQTrivia questions; in doing so, we explored challenges around model selection, feature development, and overtraining. In exploring these topics, I use many of the same strategies as in my teaching. When we make decisions, I use questions to guide students through evaluating different experimental methods or possible system designs. And I hold mentoring evaluation sessions at the end of each semester to get feedback about what worked well over the semester – and what did not! By working with students over longer periods, I've learned how mentoring approaches change as students develop. For early undergraduate students, I've learned effective ways to mentor technical skills. But for more senior students, I've learned how to mentor towards their individual career goals and seen how important it is (especially for graduate students) to feel ownership over their projects from day one.

Professional: In addition, I have mentored graduate, undergraduate students, and students in our local community college about their professional development. I consider this part of service to the university (and have been recognized, for example, with the 2019 Graduate Student Outstanding Ambassador Award) but also as part of a personally important effort towards improving the diversity and inclusion of computer science.

In both my teaching and mentoring, I ensure students know that errors are a very natural part of the learning process, and also that they will need to continue learning from these errors for the rest of their careers. I believe this helps all students in the classroom (from those struggling to the highest achievers), but also helps them mature as computer scientists – to become people who can continue to grow as technology changes and develops throughout their careers.